

SQL + Python/Pandas + Power BI Interview Questions & Answers

Bilingual Interview Notes — English + Hindi (Roman Hindi) | Examples + Syntax

1. Introduction / Project Explanation

Interview answer: I am a data-focused professional working with Python, Pandas, SQL and Power BI. My work involves collecting data, cleaning and transforming it, querying databases, building data models, and creating dashboards for business reporting.

Hindi: Main data ke saath kaam karta/karti hoon. Python/Pandas se data clean aur transform, SQL se data query, aur Power BI se modelling aur dashboard/reporting karta/karti hoon.

Project structure:

- Coding languages: Python and SQL.
- Tools: Jupyter/Google Colab, MySQL, Power BI, Power Query and APIs.
- Typical flow: Source/API/CSV → cleaning & transformation → SQL/data model → Power BI dashboard.

2. SQL — Normalization

What is normalization?

English: Normalization is the process of organizing data into related tables to reduce data redundancy and avoid insert, update and delete anomalies.

Hindi: Normalization ka matlab data ko logically related tables me organize karna hai taaki duplicate data kam ho aur update/insert/delete problems na aaye.

1NF — First Normal Form

Each column should contain atomic/single values and there should be no repeating groups.

Bad:
Customer | Phone
1 | 9876, 8765

Good:
Customer | Phone
1 | 9876
1 | 8765

Hindi: Ek cell me multiple values nahi honi chahiye.

2NF — Second Normal Form

Table must be in 1NF and every non-key column must depend on the whole primary key. This matters especially with composite keys.

OrderDetail(order_id, product_id, product_name, qty)
If PK = (order_id, product_id), product_name depends only on product_id.
Move product_name to Product(product_id, product_name).

Hindi: Non-key column ko composite key ke poore combination par depend karna chahiye.

3NF — Third Normal Form

Table must be in 2NF and non-key columns should not depend on another non-key column.

Employee(emp_id, dept_id, dept_name)
dept_name depends on dept_id, not directly on emp_id.
Use:
Employee(emp_id, dept_id)
Department(dept_id, dept_name)

Hindi: Non-key column kisi doosre non-key column par depend nahi hona chahiye.

3. SQL — Indexing

English: An index is a data structure used to speed up data retrieval. It is commonly created on columns used in WHERE, JOIN, ORDER BY and sometimes GROUP BY. Indexes improve reads but use storage and can slow INSERT/UPDATE/DELETE because the index also needs maintenance.

Hindi: Index search ko fast banata hai, lekin extra storage leta hai aur data change hone par maintain karna padta hai.

Types / MySQL clarification

- **Clustered index:** In MySQL InnoDB, the primary key is the clustered index; table rows are organized around the primary key. A table has one clustered index.
- **Secondary (non-clustered) index:** Other indexes such as UNIQUE or normal indexes. A table can have multiple secondary indexes.
- **Unique index:** Prevents duplicate indexed values (subject to NULL behavior).
- **Composite index:** Index on multiple columns, e.g. (customer_id, order_date).

```
CREATE INDEX idx_orders_customer
ON Orders(customer_id);
```

```
CREATE INDEX idx_orders_customer_date
ON Orders(customer_id, order_date);
```

```
CREATE UNIQUE INDEX ux_customer_email
ON Customer(email);
```

4. Primary Key vs UNIQUE Constraint

Both can enforce uniqueness, but they are not identical.

Feature	Primary Key	UNIQUE
Duplicates	Not allowed	Not allowed
NULL	Not allowed	MySQL can allow NULL (multiple NULLs may be allowed)
Per table	Only one PK	Can have multiple UNIQUE constraints
Main use	Main row identifier	Enforce alternate/candidate uniqueness

```
CREATE TABLE Customer(
customer_id INT PRIMARY KEY,
email VARCHAR(100) UNIQUE
);
```

Interview line: Primary key uniquely identifies each row and cannot be NULL. UNIQUE also prevents duplicates, but a table can have multiple UNIQUE constraints and MySQL permits NULL values in a UNIQUE column.

5. Foreign Key

A foreign key links a child table to a parent table and helps maintain referential integrity.

```
CREATE TABLE Customer(
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100)
);
```

```
CREATE TABLE Orders(
order_id INT PRIMARY KEY,
customer_id INT,
FOREIGN KEY (customer_id)
REFERENCES Customer(customer_id)
);
```

6. CTE vs Subquery

CTE

CTE (Common Table Expression) is a temporary named result set defined using WITH. It improves readability and can be reused within the same SQL statement.

```
WITH sales AS (  
  SELECT customer_id, SUM(amount) AS total_sales  
  FROM Orders  
  GROUP BY customer_id  
)  
SELECT *  
FROM sales  
WHERE total_sales > 10000;
```

Hindi: CTE query ke andar ek temporary named result jaisa hota hai. Complex query ko readable banata hai.

Subquery

A subquery is a query written inside another query.

```
SELECT *  
FROM Customer  
WHERE customer_id IN (  
  SELECT customer_id  
  FROM Orders  
);
```

Difference: CTE usually makes multi-step logic easier to read; subquery is embedded directly inside another query. Choice depends on readability and the SQL engine/optimizer.

7. CTE Scenario — Customer, Product, Orders

Important: Your listed columns do not show the keys needed to connect all three tables. Assume Orders has customer_id and product_id, and Orders.orderName exists. Product has product_id.

```
WITH order_product AS (  
  SELECT  
    o.orderName,  
    p.product_name,  
    p.category  
  FROM Orders o  
  JOIN Product p  
  ON o.product_id = p.product_id  
)  
SELECT  
  product_name,  
  category,  
  orderName  
FROM order_product;
```

Hindi: Pehle CTE me Orders ko Product se product_id par join kiya, phir required columns select kiye. Customer table tab join hogi jab customer-related column chahiye.

8. SQL Joins

INNER JOIN: Only matching rows from both tables.

```
SELECT c.customer_name, o.order_id  
FROM Customer c  
INNER JOIN Orders o  
ON c.customer_id = o.customer_id;
```

LEFT JOIN: All rows from left table + matching rows from right table.

```
SELECT c.customer_name, o.order_id  
FROM Customer c  
LEFT JOIN Orders o  
ON c.customer_id = o.customer_id;
```

Customers who never placed an order:

```
SELECT c.customer_id, c.customer_name  
FROM Customer c  
LEFT JOIN Orders o  
ON c.customer_id = o.customer_id  
WHERE o.customer_id IS NULL;
```

Hindi: LEFT JOIN se saare customers milenge; jinke order nahi hain unke order-side columns NULL honge.

9. Scenario Questions — Two Tables

Q1. Find customers with no orders

Use LEFT JOIN + IS NULL as shown above. Alternative:

```
SELECT c.customer_id, c.customer_name
FROM Customer c
WHERE NOT EXISTS (
  SELECT 1
  FROM Orders o
  WHERE o.customer_id = c.customer_id
);
```

Q2. Find customers with more than 3 orders

```
SELECT c.customer_id, c.customer_name,
COUNT(o.order_id) AS order_count
FROM Customer c
JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) > 3;
```

Q3. Find orders placed by a particular customer

```
SELECT o.order_id, o.order_date
FROM Orders o
JOIN Customer c
ON o.customer_id = c.customer_id
WHERE c.customer_name = 'Amit';
```

10. Python — Data Types

Common built-in Python data types: int, float, complex, bool, str, list, tuple, set, dict and NoneType.

```
age = 25 # int
price = 99.5 # float
name = "Amit" # str
active = True # bool
nums = [1, 2, 3] # list
point = (10, 20) # tuple
unique = {1, 2, 3} # set
person = {"name": "Amit"} # dict
value = None # NoneType
```

Interview line: Python has built-in numeric, sequence, set, mapping and boolean/None types.

11. Python Functions

Function: A reusable block of code that performs a task. User-defined functions are created using def. Predefined/built-in functions are already provided by Python.

```
# User-defined function
def add(a, b):
    return a + b

print(add(10, 20))

# Built-in / predefined functions
len("Python")
max(10, 20, 30)
sum([10, 20, 30])
type(10)
```

Hindi: Function reusable code block hai. User-defined function hum khud banate hain; built-in functions Python provide karta hai.

12. Pandas — Import and Read CSV

```
import pandas as pd

df = pd.read_csv("customers.csv")
print(df.head())
```

How to check whether the file has rows?

```
print(df.empty) # True = no rows, False = has data
print(len(df)) # number of rows
print(df.shape) # (rows, columns)
```

Hindi: df.empty simplest check hai. False aaye to DataFrame empty nahi hai.

13. If CSV Is Not Showing / Not Loading

- Check the file path and current working directory.
- Check the exact file name and extension.
- Use os.getcwd() to see the current folder.
- Check encoding if special characters are present.
- Check separator if it is not comma-separated.

```
import os
print(os.getcwd())

df = pd.read_csv("customers.csv", encoding="utf-8")
# For semicolon-separated CSV:
df = pd.read_csv("customers.csv", sep=";")
```

14. Validate Email Column in Pandas

A practical interview approach is to define a function and apply it to the email column. This example checks a basic email pattern; it is not a full guarantee that an email address exists.

```
import pandas as pd
import re

def is_valid_email(email):
    if pd.isna(email):
        return False
    pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
    return bool(re.match(pattern, str(email)))

df = pd.read_csv("customers.csv")
df["email_valid"] = df["email"].apply(is_valid_email)

print(df[["email", "email_valid"]])
```

Hindi: Function email ko check karta hai aur True/False return karta hai. apply() function ko har row ke email par chalata hai.

15. Messed-up Date Column — Pandas

Convert the column using to_datetime and handle invalid values.

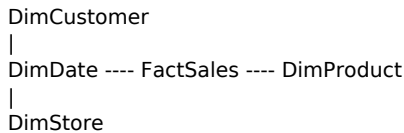
```
df["order_date"] = pd.to_datetime(
    df["order_date"],
    errors="coerce"
)

# Invalid dates become NaT
print(df["order_date"].isna().sum())
```

Interview line: I use pd.to_datetime with errors='coerce' so invalid date values become NaT, which I can then identify and clean.

16. Power BI — Star Schema

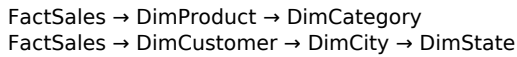
Star schema: One central fact table connected directly to multiple dimension tables. It is simple and commonly preferred for BI reporting.



Hindi: Beech me fact table aur around me dimension tables hoti hain, isliye star jaisa structure dikhta hai.

17. Snowflake Schema

Snowflake schema normalizes dimension tables into additional related tables.



Difference: Star is simpler and often easier for reporting; snowflake reduces dimension redundancy but adds more joins.

18. Fact vs Dimension Tables

- Fact table: Stores measurable business events such as sales amount, quantity, order count.
- Dimension table: Stores descriptive attributes such as customer, product, date, region.
- Example: FactSales(sale_id, customer_id, product_id, date_id, quantity, sales_amount);
DimProduct(product_id, product_name, category).

19. Power BI Data Modelling

Data modelling means creating tables and relationships so Power BI can correctly filter, aggregate and analyze data.

Hindi: Tables ke beech proper relationships banana aur fact/dimension structure ko organize karna data modelling hai.

20. Cardinality / Relationships

- One-to-many (1:*) : One customer can have many orders. This is the common star-schema relationship.
- Many-to-one (*:1) : Same relationship viewed from the other side.
- One-to-one (1:1) : One row maps to one row.
- Many-to-many (*:*) : Multiple rows can match multiple rows; use carefully and consider a bridge table.

Recommended interview answer: In a star schema, dimensions are usually on the '1' side and facts on the '*' side.

21. Cross Filter Direction — Single vs Both

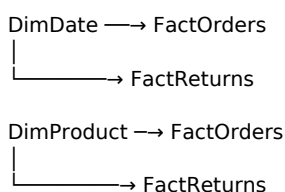
Single: Filter flows in one direction, usually Dimension → Fact. This is generally the safest/default design for a star schema.

Both: Filter can propagate in both directions. It can solve specific scenarios but may create ambiguity or unexpected filtering in complex models.

Hindi: Single me filter ek direction me flow karta hai; Both me dono directions me flow kar sakta hai.

22. Can Power BI Have Two Fact Tables?

Yes. A Power BI model can have multiple fact tables. Prefer shared/conformed dimensions rather than directly connecting fact tables when possible.



Hindi: Do fact tables ho sakti hain, jaise Orders aur Returns. Dono ko common Date/Product dimensions se connect karna better design hai.

23. Scenario — Orders + Returns

Question: Dashboard me Orders aur Product use ho raha hai. Kya Return Orders ke basis par Product/Order visual ko filter kar sakte hain agar cross-filter direction Both kar dein?

Best interview answer: Technically, changing relationships to Both may allow filter propagation if the relationship path is valid and unambiguous. But simply turning everything to Both is not the recommended design. Use shared dimensions and measures; if the required filter must travel from Returns to Product, design a proper relationship/filter path or use a measure-based approach.

Hindi: Sirf Both kar dena best practice nahi hai. Orders aur Returns ko common Product/Date dimensions se connect karke model banana better hai. Zarurat par measure se return-related filter apply kar sakte hain.

24. Power BI Measures — What and Why?

A measure is a DAX calculation evaluated at query time according to the current filter context. Measures are ideal for dynamic KPIs and aggregations.

```
Total Sales =  
SUM(FactSales[SalesAmount])
```

```
Total Quantity =  
SUM(FactSales[Quantity])
```

```
Average Sales =  
AVERAGE(FactSales[SalesAmount])
```

Why use measures? They are dynamic, respond to slicers/filters, and avoid storing a separate calculated result for every row.

25. Calculated Column

A calculated column is calculated row by row and stored in the model.

```
SalesAmount =  
FactSales[Quantity] * FactSales[UnitPrice]
```

Measure vs calculated column: Column = row-level stored value; Measure = dynamic calculation based on filter context.

Hindi: Column har row ke liye calculate hota hai aur model me store hota hai. Measure visual/filter context ke according dynamically calculate hota hai.

26. DAX CALCULATE

CALCULATE evaluates an expression in a modified filter context.

```
Total Sales =  
SUM(FactSales[SalesAmount])
```

```
Online Sales =  
CALCULATE(  
[Total Sales],  
FactSales[Channel] = "Online"  
)
```

Hindi: CALCULATE existing filter context ko modify karke calculation karta hai.

27. SUM vs SUMX

SUM: Adds values from one numeric column.

```
Total Sales = SUM(Sales[SalesAmount])
```

SUMX: Iterates row by row over a table/expression, calculates an expression for each row, then sums the results.

```
Total Sales =
SUMX(
Sales,
Sales[Quantity] * Sales[UnitPrice]
)
```

Hindi: SUM direct column ka total karta hai. SUMX har row par expression calculate karke sabko add karta hai.

28. Can SUM and SUMX Be Used Inside CALCULATE?

Yes. Both can be used as the expression inside CALCULATE.

```
Sales 2026 =
CALCULATE(
SUM(Sales[SalesAmount]),
Sales[Year] = 2026
)
```

```
Sales 2026 by Row =
CALCULATE(
SUMX(
Sales,
Sales[Quantity] * Sales[UnitPrice]
),
Sales[Year] = 2026
)
```

29. Fetch Data Through API in Power BI

Power BI Desktop: Home → Get Data → Web → enter API URL → authenticate if required → Power Query transforms the response → Load to model.

Example API URL:
<https://api.example.com/sales>

Typical Power Query M:
let
Source = Json.Document(
Web.Contents("https://api.example.com/sales")
)
in
Source

Important: Real APIs may require headers, API keys/OAuth, pagination and privacy/credential settings. Never hard-code sensitive credentials into a shared report.

30. Quick Interview Revision — One-line Answers

- Normalization: Organize relational data to reduce redundancy and anomalies.
- Index: Data structure that speeds up retrieval but adds storage/maintenance cost.
- CTE: Named temporary result set created with WITH for a SQL statement.
- Join: Combines rows from tables using a related condition.
- Primary key: Unique, non-NULL row identifier; one per table.
- Foreign key: Column(s) referencing a key in another table.
- Measure: Dynamic DAX calculation evaluated in filter context.
- Calculated column: Row-by-row DAX value stored in the model.
- Star schema: One fact table surrounded by dimensions.
- Snowflake schema: Normalized dimensions with additional related tables.
- Cardinality: Defines how many rows can relate between tables.
- Single cross-filter: Filter flows one way; Both: filter can flow both ways.
- SUM: Adds a column; SUMX: iterates rows and sums an expression.

- Pandas import: `import pandas as pd.`
- CSV read: `pd.read_csv('file.csv').`
- Empty DataFrame check: `df.empty.`
- Date cleaning: `pd.to_datetime(df['date'], errors='coerce').`
- Email validation: define a validation function and use `df['email'].apply(function).`

31. Mini Project Explanation Template

English: In my project, I worked on a data pipeline/reporting solution. Data came from CSV/API/database sources. I used Python and Pandas for data validation and cleaning, SQL for querying and transformation, and Power BI for data modelling, DAX measures and dashboards. I designed a star-schema style model with fact and dimension tables, created appropriate relationships, and built KPIs for business analysis.

Hindi: Mere project me data CSV/API/database se aaya. Python aur Pandas se validation/cleaning, SQL se querying/transformation aur Power BI se modelling, DAX measures aur dashboard banaya. Fact aur dimension tables ke saath star-schema based model use kiya.

32. Interview Tip

For scenario questions, answer in this order: 1) Understand the business requirement → 2) Identify tables and keys → 3) Choose relationship/join → 4) Write query/DAX → 5) Explain why the approach is safe and scalable.

Hindi: Scenario question me pehle requirement samjho, phir keys/tables identify karo, join/relationship choose karo, query/DAX likho aur last me reason explain karo.